



Project Description

Traditional fitness applications tend to act as digital notebooks, simply recording data without actively providing users with insights and analytics. The burden of calculating progressive overload, recovery, and other aspects, is placed entirely on the user. Without a systematic and consistent approach to managing their progress, many athletes encounter “plateauing”, where they go a long period without improving. Users are constrained by busy schedules and a lack of athletic science knowledge, frequently leading to missed or inefficient workout sessions. Pre-planned routines can be disrupted and result in inconsistent data that many typical users would be unable to adjust to themselves.

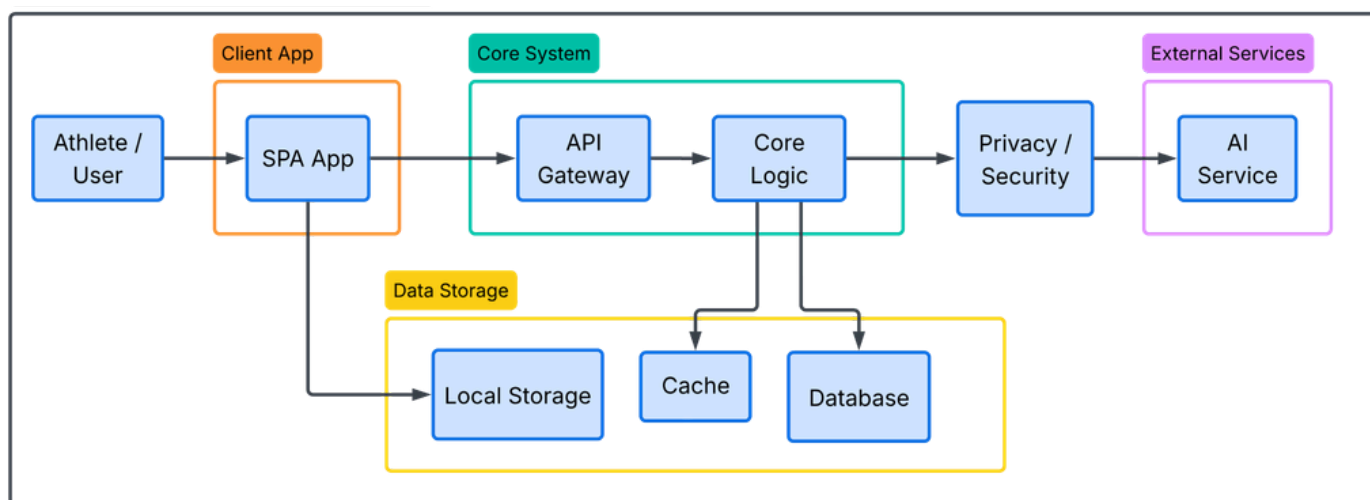
OptiLifts is designed to transform the traditional, manual approach to strength training into an intelligent, automatic experience for the user. The system addresses the “athletic plateau” problem by utilising the user’s historical data, guiding the user through personalised, and optimised training programs. The load on the user to manage their own results, and compute the complex logic behind progressive overload is reduced.

The system does not just track reps and sets, but also is context-aware. It adapts to the disruptions of life. By dynamically adjusting sessions, depending on the user’s schedule and time constraints. Whether a user misses a session, or only has 20 minutes to train, the system will adapt the exercise priority, ensuring users are working out as optimally as possible, avoiding the “plateau” and ensuring their workouts cover all important muscle groups. The system will also ensure to take the user's recovery needs into account by integrating a ‘rate of perceived exertion’, which can adapt a session in real time to prevent injury and burnout.



High Level System Overview

Optilifts is a smart strength-training platform built to automate progressive overload. The system uses a Single Page Application (SPA) that talks to a secure, cloud-hosted API. The system is organised into four main areas:



1. Client Presentation Layer (App Interface)

This is the primary screen the athlete interacts with. It features a workout mode designed for use during training sessions where users can track sets, record how difficult a lift felt (RPE), and create workout templates.

2. API Gateway and Logic Layer

This is the brain of the system. It manages security and runs the core tools of the applications, including a workout builder to handle the logic behind complex routines, a progression engine to analyse workouts and suggest increments, and a dynamic schedule that automatically moves or re-prioritises missed workouts.

3. Persistence Layer (Data Storage)

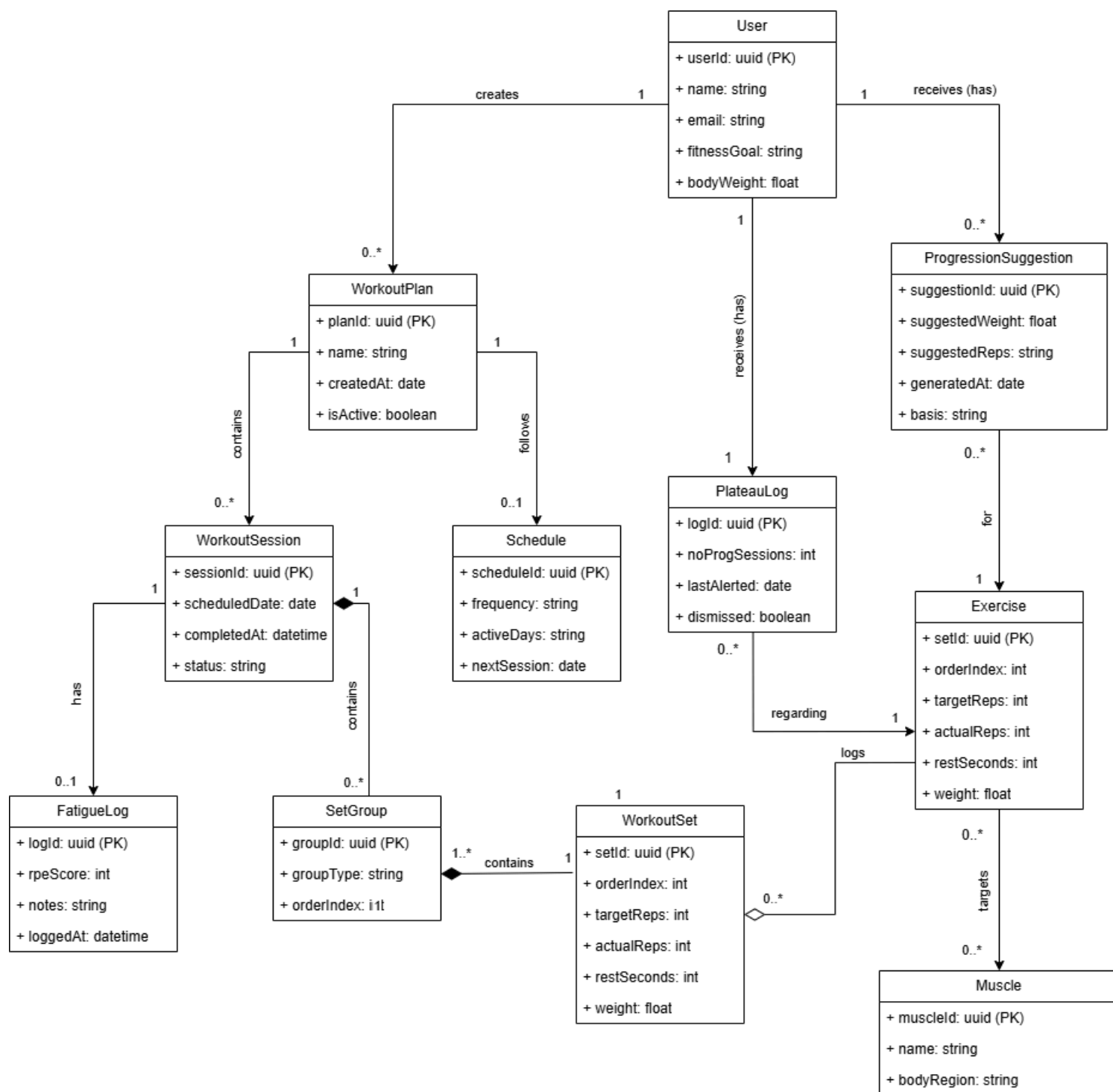
This layer stores and keeps data safe and fast to access. A database stores permanent records like historical workout data and profile information, and caching is used to speed up the app and reduce costs of AI features. Local storage is available for when users want to workout offline.

4. External Services (AI and Security)

Optilifts uses AI integration to compute plateau detection and a security layer to remove all personal information to ensure the privacy of users is protected and compliant with POPIA.



Domain Model



→
This arrow is used as an association with direction - for the purpose of readability.



Technology stack

Component	Technology	Justification
Frontend	React (via Next.js)	Fulfils SPA requirements. Component-based, highly-response and clean UI .
Design System	Shadcn.ui & Tailwind CSS	Customisable UI that ensures lightweight and responsive app.
Backend	Node.js (Next.js API)	High-performance framework that handles both frontend and API routes. Integrates well with Azure App Service.
Database	PostgreSQL	Industry-standard relational database for storing complex hierarchical workout data , and training history.
Caching	Redis	High-speed retrieval of session data and for caching AI responses to minimise token costs and improve latency.
Cloud Hosting	Microsoft Azure	Centralises services under Azure for Students sponsorship.
AI Engine	Azure OpenAI	Access to GPT-4o/o1-mini for progression engine and plateau detection logic.
CI/CD	GitHub Actions	Automates testing and deployment pipeline, ensuring every push to the main branch automatically deployed to Azure
IaC	Terraform	Define entire infrastructure as code, ensuring a reproducible and professional environment



Development Approach

Methodology

We will follow the **Agile methodology with 2-week sprint cycles**. We have found this works best as we are able to continuously deliver working features, adapt to client and mentor feedback quickly and readjust our strategy when needed. This is especially helpful with Optilifts as it allows us to release working increments of the core workout logic early, surface flawed assumptions, and refine the product iteratively.

Team structure

Each person will lead a certain domain however everyone will still develop in other areas. The proposed roles are Team lead, Frontend lead, Backend lead, Devops lead and the Quality assurance lead.

Communication plan

Internal communication: Communication will occur through in-person and online meetings. Online meetings will be held on Discord.

Task management: All work will be tracked on GitHub project boards. This will allow us to know what everyone is working on and the timeframe of respective tasks.

Client Communication: The team lead will be the main point of contact with the industry partner.

Meeting cadence

Sprint Planning: Held on the first day of the sprint. We will define the sprint goal, break down client requirements, and assign GitHub issues.

Mid-Sprint Sync: Held halfway through a sprint. A check-in to see what was done, what is next, and if anyone is stuck.

Sprint Review: Held at the end of the sprint. We present the software increment to the client and mentor for feedback.

Sprint Retrospective: A quick internal review to see how we can improve our team speed and workflow for the next sprint.